

PROGRAMACIÓN PARALELA · ANÁLISIS TÉCNICO

Descomposición de datos

Dónde y cómo se reparte el trabajo entre hilos en VentasParalelo, y por qué la segunda descomposición —la del resultado— es la que decide el rendimiento.

Proyecto: VentasParalelo · motor de agregación paralela en .NET 8

Alcance: técnicas de descomposición aplicadas, con referencias al código fuente

Repositorio: github.com/IsraelDev-var/VentasParalelo

1. Qué es descomponer un problema

El marco teórico, en dos párrafos.

Paralelizar un programa empieza siempre por la misma pregunta: **¿en qué unidades independientes se puede partir este trabajo?** A esas unidades se les llama tareas, y al procedimiento de identificarlas, **descomposición**.

La literatura clásica distingue varias técnicas según la naturaleza del problema:

Técnica	Cuándo se aplica	¿En este proyecto?
De datos	Los datos se parten y todas las tareas aplican la misma operación a su porción.	Sí — es la base de todo
Recursiva	Divide y vencerás: el problema se parte en subproblemas del mismo tipo.	Sí — en la reducción en árbol
Exploratoria	Se busca en un espacio de soluciones; al encontrar una, el resto puede abandonar.	No
Especulativa	Se ejecutan ramas por adelantado, antes de saber cuál hará falta.	No

Las dos últimas no aparecen aquí, y no por descuido: en VentasParalelo el trabajo es **fijo y conocido de antemano**. Hay que visitar los cinco millones de registros, ni uno más ni uno menos. No hay nada que buscar, ninguna rama que adivinar y ninguna razón para terminar antes de tiempo. La sección 5 detalla esta comprobación.

La distinción que conviene tener clara

Descomposición es decidir *qué tareas existen*. **Mapeo** es decidir *qué hilo ejecuta cada tarea y cuándo*. Son cosas distintas: el chunking dinámico de este proyecto no es otra descomposición, es la misma descomposición de datos con un mapeo dinámico en lugar de estático.

2. Dónde se decide el reparto

La capa de particionado: el lugar exacto donde ocurre la descomposición.

El proyecto aísla la descomposición en su propia capa, separada del código que procesa los datos. Esa separación es deliberada: permite ejecutar **exactamente el mismo algoritmo de agregación** sobre dos repartos distintos y atribuir cualquier diferencia de rendimiento únicamente al reparto.

```
src/VentasParalelo.Core/Partitioning/ContiguousRangePartitioner.cs
```

Aquí nacen las particiones

```
var particiones = new List<Range>(partitionCount);
var baseSize = length / partitionCount;
var remainder = length % partitionCount;
var start = 0;

for (var i = 0; i < partitionCount; i++)
{
    var size = baseSize + (i < remainder ? 1 : 0);
    var end = start + size;
    particiones.Add(new Range(start, end));
    start = end;
}

return particiones;
```

Con 1.000.000 de filas y 4 hilos, el resultado son cuatro rangos:

```
[0, 250000) [250000, 500000) [500000, 750000) [750000, 1000000)
```

Dos detalles que hacen buena a esta función

No se copia ni una sola fila. Lo que devuelve son cuatro objetos `Range`, que son dos enteros cada uno. Los datos permanecen intactos en el arreglo original; lo que se reparte son los *índices*. Si el particionador copiara los datos, el costo de repartir superaría al de procesar.

El reparto queda parejo hasta la última fila. Cuando la división no es exacta, la línea `baseSize + (i < remainder ? 1 : 0)` entrega una fila extra a las primeras particiones. Así, la diferencia entre la partición más grande y la más chica nunca pasa de una fila. Importa porque el tiempo total lo marca el hilo que más tarda: un reparto desbalanceado deja hilos ociosos esperando al rezagado.

3. Las tres formas de repartir

Misma descomposición, tres criterios distintos — y rendimientos muy distintos.

3.1 Contigua — bloques consecutivos

Cada hilo recibe un bloque seguido del arreglo.



Es el reparto que mejor aprovecha la **caché de la CPU**: cuando un hilo pide la fila 7, el procesador trae a caché todo el bloque de memoria que la rodea, y las filas 8, 9 y 10 ya están listas cuando le tocan. Recorrer memoria en orden es el patrón que el hardware premia.

3.2 Cíclica (round-robin) — filas alternadas

El hilo p recibe los índices p , $p+P$, $p+2P$, ...



src/VentasParalelo.Core/Partitioning/RoundRobinPartitioner.cs

```
for (var p = 0; p < partitionCount; p++)
{
    var count = baseSize + (p < remainder ? 1 : 0);
    particiones.Add(new StridedRange(p, partitionCount, count));
}
```

En lugar de un rango, guarda una tripleta: *empieza en p , avanza de P en P , tantas veces*. Por eso `StridedRange` tiene un campo `Step` que `Range` no necesita.

Reparte igual de bien y aun así rinde peor

Cada hilo recibe exactamente la misma cantidad de filas que en el reparto contiguo — el balance de carga es idéntico. Sin embargo, mide entre 19 y 27 puntos porcentuales menos de eficiencia. Como el algoritmo de acumulación es el mismo, esa diferencia aísla un efecto puro de **localidad de caché**: saltar por la memoria cuesta más que recorrerla en orden. Esta estrategia está en el proyecto justamente para poder medir ese efecto por separado.

3.3 Dinámica (chunking) — trozos por demanda

El arreglo se corta en muchos trozos pequeños y cada hilo toma el siguiente disponible en cuanto termina el anterior. El reparto no se decide de antemano:



Hilos: ■ H0 ■ H1 ■ H2 ■ H3 · ejemplo con 24 filas y 4 hilos

```
src/VentasParalelo.Core/Aggregation/DynamicChunkReductionAggregator.cs
```

```
var chunkSize = Math.Max(1, Math.Min(_tamanoChunk, registros.Length));  
var particionador = Partitioner.Create(0, registros.Length, chunkSize);
```

En el proyecto los trozos son de 4.096 filas. Con 5 millones de filas eso da unos 1.220 trozos repartidos entre 8 hilos — muchos más trozos que hilos, que es la condición para que el balanceo dinámico funcione. La ventaja aparece cuando el costo por fila es irregular: un hilo que agarra filas baratas simplemente vuelve por más. El precio es más coordinación entre hilos.

Criterio	Contigua	Cíclica	Dinámica
Localidad de caché	Excelente	Mala	Buena
Balance si el costo es irregular	Rígido	Rígido	Se adapta
Costo de coordinación	Nulo	Nulo	Cola compartida
Cuándo decide el reparto	Antes de empezar	Antes de empezar	Durante la ejecución

4. La segunda descomposición

La que casi nadie nota, y la que realmente decide el rendimiento.

Todo lo anterior descompone la **entrada**. Pero el proyecto descompone además la **salida**, y esa decisión —no el reparto de filas— es la que separa un 5 % de eficiencia de un 85 %.

La instrucción cabe en una línea. Está en el `localInit` de `Parallel.ForEach`:

`src/VentasParalelo.Core/Aggregation/LocalReductionAggregator.cs`

```
Parallel.ForEach(
    particiones,
    new ParallelOptions { MaxDegreeOfParallelism = maxDegreeOfParallelism },
    localInit: () => new AggregationResult(),
    body: (rango, _, local) =>
    {
        var mapeo = Stopwatch.StartNew();
        for (var i = rango.Start.Value; i < rango.End.Value; i++)
            SequentialAggregator.Acumular(local, in registros[i]);

        local.FilasProcesadas += rango.End.Value - rango.Start.Value;
        mapeo.Stop();
        diag.SumarMapeo(mapeo.Elapsed);
        return local;
    },
    localFinally: local =>
    {
        // El merge solo ocurre una vez por particion (no una vez por fila), así que
        // el lock aquí casi no compite: por eso no se mide como "contencion" aparte,
        // sino como parte del costo fijo de la reduccion.
        var reduccion = Stopwatch.StartNew();
        lock (mergeLock)
        {
            resultado.MergeFrom(local);
        }
        reduccion.Stop();
        diag.SumarReduccion(reduccion.Elapsed);
    });
```

Cada hilo no solo *lee* un pedazo distinto de los datos: también *escribe* en un acumulador distinto. Al no compartir el destino de la escritura, no hay nada que sincronizar mientras procesa.

La comparación que lo demuestra

Estas dos estrategias descomponen la entrada de forma **idéntica** — ambas llaman al mismo `ContiguousRangePartitioner`. Lo único que cambia es si descomponen o no el resultado:

Sin descomponer la salida

```
LockDictionaryAggregator  
  
var result = new AggregationResult();  
var sync    = new object();  
  
// un solo acumulador  
// para los 8 hilos  
// → hace falta un candado  
// → 5.000.000 de bloqueos
```

Descomponiendo la salida

```
LocalReductionAggregator  
  
localInit: () =>  
    new AggregationResult()  
  
// un acumulador  
// por hilo  
// → sin candado al procesar  
// → 8 fusiones al final
```

	lock sobre Dictionary	Acumuladores locales
Descomposición de la entrada	Contigua	Contigua (la misma)
Descomposición de la salida	Ninguna	Una por hilo
Sincronizaciones con 5 M de filas	5.000.000	8
Eficiencia con 8 hilos	5 %	85 %

Por qué es válido separar y luego sumar

Este patrón funciona porque la operación que se aplica —la suma— es **asociativa y conmutativa**: da igual en qué orden se sumen los subtotales, el total es el mismo. Es la propiedad matemática que autoriza a cada hilo a acumular por su cuenta.

La fusión de los resultados parciales

```
public void MergeFrom(AggregationResult otro)  
{  
    foreach (var (clave, valor) in otro.MontoPorSucursal)  
        MontoPorSucursal[clave] = MontoPorSucursal.GetValueOrDefault(clave) + valor;  
  
    foreach (var (clave, valor) in otro.UnidadesPorProducto)  
        UnidadesPorProducto[clave] = UnidadesPorProducto.GetValueOrDefault(clave) + valor;  
  
    FilasProcesadas += otro.FilasProcesadas;  
}
```

Si la operación no tuviera esa propiedad —por ejemplo, "la primera venta registrada de cada sucursal", que depende del orden— este patrón daría un resultado incorrecto y habría que replantear la descomposición.

5. Descomposición recursiva

La segunda técnica presente, en un solo lugar del proyecto.

La reducción en árbol aplica divide y vencerás sobre los resultados parciales: parte el conjunto en dos mitades, resuelve cada una **recursivamente y en paralelo**, y combina.

src/VentasParalelo.Core/Aggregation/HierarchicalReductionAggregator.cs

```
private static AggregationResult ReducirEnArbol(
    AggregationResult[] locales, int inicio, int fin, AggregationDiagnostics diag)
{
    var cantidad = fin - inicio;
    if (cantidad == 1)
        return locales[inicio];

    var medio = inicio + cantidad / 2;

    AggregationResult izquierda = null!;
    AggregationResult derecha = null!;
    Parallel.Invoke(
        () => izquierda = ReducirEnArbol(locales, inicio, medio, diag),
        () => derecha = ReducirEnArbol(locales, medio, fin, diag));

    var reduccion = Stopwatch.StartNew();
    izquierda.MergeFrom(derecha);
    reduccion.Stop();
    diag.SumarReduccion(reduccion.Elapsed);

    return izquierda;
}
```

Los tres elementos que la identifican como descomposición recursiva están a la vista: un **caso base** (cantidad == 1 devuelve la partición tal cual), una **división** del problema en dos subproblemas del mismo tipo, y una **combinación** de los resultados al volver de la recursión.

La ganancia teórica es de profundidad: fusionar P resultados uno por uno bajo un candado son O(P) pasos en serie; en árbol son O(log P) niveles, y dentro de cada nivel las fusiones ocurren en paralelo. Con 8 particiones son 3 niveles en vez de 8 pasos.

Lo que la medición dijo al respecto

En la práctica esta estrategia rinde igual que la fusión secuencial (88 % contra 85 %, dentro del margen de ruido). La razón es que con solo 8 particiones y diccionarios de 8 y 12 claves, la fusión es tan barata que optimizarla no mueve la aguja. La ventaja del árbol se manifestaría con muchas más particiones que hilos.

Las técnicas que no están, y la comprobación

La descomposición **exploratoria** requiere un espacio de búsqueda y la posibilidad de abandonar el trabajo restante al encontrar una solución. La **especulativa** requiere ejecutar ramas antes de saber cuál se necesita. Ninguna aplica aquí: cada fila debe procesarse siempre, y no hay ramas condicionales que adivinar.

La comprobación es directa sobre el código: no existe en todo el proyecto ninguna llamada a `ParallelLoopState.Break()`, `ParallelLoopState.Stop()` ni ningún `CancellationToken` — los mecanismos con los que .NET permite terminar un bucle paralelo antes de tiempo. Todos los `.Stop()` que aparecen son de `Stopwatch`, es decir, cronómetros de medición.

Cómo se podría incorporar

Encajaría de forma natural agregando un comando de búsqueda sobre el mismo dataset —por ejemplo, localizar la primera transacción que supere cierto monto—: cada hilo buscaría en su partición y, al encontrarla, invocaría `ParallelLoopState.Stop()` para que los demás abandonen. Ahí sí aparecería el rasgo característico de la técnica: el tiempo dependería de *dónde* esté el resultado, no solo del tamaño del dataset.

6. Mapa de referencia

Dónde encontrar cada cosa en el código.

Qué	Archivo	Línea
Reparto en bloques contiguos	Partitioning/ContiguousRangePartitioner.cs	17–30
Reparto cíclico (round-robin)	Partitioning/RoundRobinPartitioner.cs	24–28
Generación de índices con paso	Partitioning/StridedRange.cs	12–20
Reparto dinámico por trozos	Aggregation/DynamicChunkReductionAggregator.cs	37
Descomposición del resultado	Aggregation/LocalReductionAggregator.cs	30
Fusión de resultados parciales	Models/AggregationResult.cs	44–53
Descomposición recursiva	Aggregation/HierarchicalReductionAggregator.cs	48–69
Caso sin descomposición del resultado	Aggregation/LockDictionaryAggregator.cs	20–21

En una frase

La conclusión del análisis

VentasParalelo aplica descomposición de datos en dos niveles: reparte la **entrada** entre los hilos —de tres formas distintas, para poder compararlas— y reparte también el **resultado intermedio**, dándole a cada hilo su propio acumulador. La primera descomposición es la evidente; la segunda es la que convierte un 5 % de eficiencia en un 85 %.